



DSA

Module 6

Matrix

Author
Srinivas Dande



My **JAVA**
LEARNING
CENTER



6. Multi Dimensional Arrays

- ♦ Multi Dimensional Array is an Array of Arrays.

Single Dimensional Array	Collection of Actual value
Two Dimensional Array	Collection of Single Dimensional Arrays
Three Dimensional Array	Collection of 2-D Arrays

6.1. Two Dimensional Arrays

- ♦ Two Dimensional Arrays can be called as 2-D array.
- ♦ 2-D array is Array of 1-D Arrays.
- ♦ 2-D Array can be called as Matrix.
- ♦ **A matrix is a collection of numbers arranged into a fixed number of rows and columns.**

6.2. 2-D Array Declaration and Construction

Syntax:

```
<dataType> <refVarName>[][] = new <dataType>[<size1>][<size2>];  
<dataType>[][] <refVarName> = new <dataType>[<size1>][<size2>];
```

Here

<size1> represents number of arrays to construct.

<size2> represents number of elements required for each array.

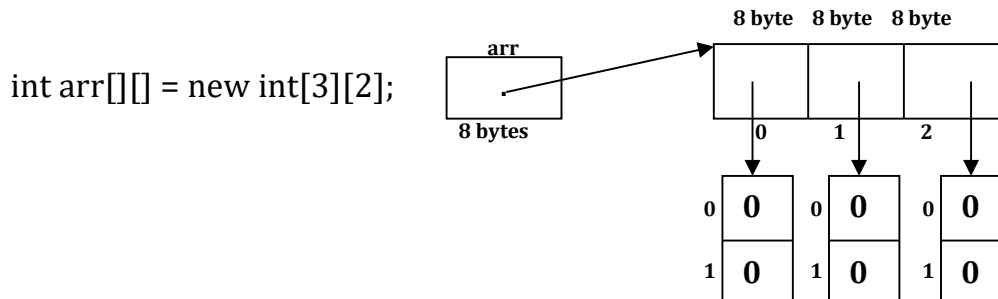
Ex:

```
int arr[][] = new int[3][2];  
int[][] arr = new int[3][2];
```



6.3. 2-D Array Initialization

- ♦ You can initialize array elements with your own value using index representation.

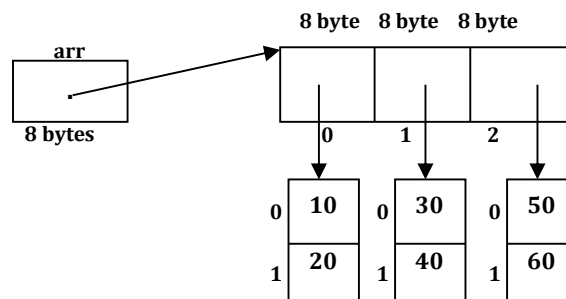


- ♦ Here we are creating 3 arrays with size 2, and these three arrays addresses will be stored in main array.

`arr[0][0]` -> Represents the 1st element of 1st array which contains value 0.
`arr[0][1]` -> Represents the 2nd element of 1st array which contains value 0.
`arr[1][0]` -> Represents the 1st element of 2nd array which contains value 0.
`arr[1][1]` -> Represents the 2nd element of 2nd array which contains value 0.
`arr[2][0]` -> Represents the 1st element of 3rd array which contains value 0.
`arr[2][1]` -> Represents the 2nd element of 3rd array which contains value 0.

- ♦ You can initialize the array elements as follows:

```
arr[0][0]    =10;  
arr[0][1]    =20;  
arr[1][0]    =30;  
arr[1][1]    =40;  
arr[2][0]    =50;  
arr[2][1]    =60;
```





- ♦ **Accessing the elements after initialization**

arr[0][0] -> Represents the 1st element of 1st array which contains value 10.
arr[0][1] -> Represents the 2nd element of 1st array which contains value 20.
arr[1][0] -> Represents the 1st element of 2nd array which contains value 30.
arr[1][1] -> Represents the 2nd element of 2nd array which contains value 40.
arr[2][0] -> Represents the 1st element of 3rd array which contains value 50.
arr[2][1] -> Represents the 2nd element of 3rd array which contains value 60.

6.4. Jagged Arrays

Ex:

```
int arr[][] = new int[3][]; //Second size is Optional
```

```
arr[0]= new int[3];
```

```
arr[1]= new int[4];
```

```
arr[2]= new int[5];
```

- ♦ Now You can 12 Elements in the Array
- ♦ This Array is called as **Jagged Array**.

6.5. Array Declaration, Construction and Initialization

Syntax:

```
<dataType> <refVarName> [][] = {{v1,v2,...},{v1,v2,...},{v1,v2,...},...};
```

Ex:

```
int arr[][]={{10,20},{30,40},{50,60}};
```

```
int arr[][]={{10,20,30},{40},{50,60,70,80},{90,100}};
```



6.6. Examples on 2-D Arrays

Lab1.java

```
package com.jlcindia.matrices;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab1 {
    public static void main(String[] args) {

        //1.Create the 2-D Array
        int mat[][] = new int[3][5];

        //2. No of Rows
        int rows = mat.length;
        System.out.println(rows);

        int r1size= mat[0].length;
        System.out.println(r1size);

        int r2size= mat[1].length;
        System.out.println(r2size);

        int r3size= mat[2].length;
        System.out.println(r3size);
    }
}
```



Lab2.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab2 {
    public static void main(String[] args) {

        // 1.Create the 2-D Array
        int mat[][] = new int[3][3];

        // Assign elements to 1st Row
        mat[0][0] = 11;
        mat[0][1] = 12;
        mat[0][2] = 13;

        // Access 1st Row elements
        for(int i=0;i<mat[0].length;i++) {
            System.out.print(mat[0][i]+"\\t");
        }

        System.out.println("\\n-----");

        // Assign elements to 2nd Row
        mat[1][0] = 14;
        mat[1][1] = 15;
        mat[1][2] = 16;

        // Access 2nd Row elements
        for(int i=0;i<mat[1].length;i++) {
            System.out.print(mat[1][i]+"\\t");
        }

        System.out.println("\\n-----");
    }
}
```



```
// Assign elements to 3rd Row
mat[2][0] = 17;
mat[2][1] = 18;
mat[2][2] = 19;

// Access 3rd Row elements
for(int i=0;i<mat[2].length;i++) {
    System.out.print(mat[2][i]+"\\t");
}
}
```

Lab3.java

```
package com.jlccindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab3 {

    //Time Complexity - O( m * n)
    // where m - rows and n - cols
    static void printMatrix(int mat[][]) {

        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                System.out.print(mat[i][j] + "\\t");
            }
            System.out.println("");
        }
    }
}
```



```
public static void main(String[] args) {  
  
    // 1.Create the 2-D Array  
    int mat[][] = new int[3][3];  
  
    printMatrix(mat);  
  
    // Assign elements to 1st Row  
    mat[0][0] = 11;  
    mat[0][1] = 12;  
    mat[0][2] = 13;  
  
    // Assign elements to 2nd Row  
    mat[1][0] = 14;  
    mat[1][1] = 15;  
    mat[1][2] = 16;  
  
    // Assign elements to 3rd Row  
    mat[2][0] = 17;  
    mat[2][1] = 18;  
    mat[2][2] = 19;  
  
    printMatrix(mat);  
}  
}
```



Lab4.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab4 {

    //Time Complexity - O( rows * cols)
    static void printMatrix(int mat[][]) {

        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                System.out.print(mat[i][j] + "\t");
            }
            System.out.println("");
        }
    }

    public static void main(String[] args) {

        // 1.Create and Initialize the 2-D Array
        int mat[][] = {
            {11,12,13,14},
            {21,22,23,24},
            {31,32,33,34},
            {41,42,43,44}
        };

        printMatrix(mat);
    }
}
```



Lab5.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab5 {
    public static void main(String[] args) {
        // 1.Create the 2-D Array
        int arr[][] = new int[3][];

        arr[0] = new int[3];
        arr[1] = new int[4];
        arr[2] = new int[5];

        printMatrix(arr);

        // Assign elements to 1st Row
        arr[0][0] = 11;
        arr[0][1] = 12;
        arr[0][2] = 13;

        // Assign elements to 2nd Row
        arr[1][0] = 21;
        arr[1][1] = 22;
        arr[1][2] = 23;
        arr[1][3] = 24;

        // Assign elements to 3rd Row
        arr[2][0] = 31;
        arr[2][1] = 32;
        arr[2][2] = 33;
        arr[2][3] = 34;
        arr[2][4] = 35;

        printMatrix(arr);
    }
}
```



Lab6.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
package com.jlcindia.matrices;

public class Lab6 {

    //Time Complexity - O(m* n)
    //Aux Space Complexity - O(1)
    static void printMatrix(int mat[][]) {

        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                System.out.print(mat[i][j] + "\t");
            }
            System.out.println("");
        }
    }

    public static void main(String[] args) {

        // 1.Create the 2-D Array
        int arr[][] = {
            {11,12,13},
            {21,22,23,24},
            {31,32,33,34,35}
        };

        printMatrix(arr);
    }
}
```



6.7. Problems on Matrices

Problem1A.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem1- Print the Main Diagonal of n * n Matrix
public class Problem1A {

    //Time Complexity - O(n2)
    //Aux Space Complexity - O(1)
    static void mainDiagonal(int mat[][]) {

        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                if (i == j)
                    System.out.print(mat[i][j] + "\t");
            }
        }
    }

    public static void main(String[] args) {

        int mat[][] = {
            { 1, 2, 3, 4 },
            { 5, 6, 7, 9 },
            { 9, 10, 11, 12 },
            { 13, 14, 15, 16 }
        };

        mainDiagonal(mat);
    }
}
```



Problem1B.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Problem1- Print the Main Diagonal of n * n Matrix

public class Problem1B {

    //Time Complexity - O(n)
    //Aux Space Complexity - O(1)
    static void mainDiagonal(int mat[][]) {

        for (int i = 0; i < mat.length; i++) {
            System.out.print(mat[i][i] + "\t");
        }
    }

    public static void main(String[] args) {
        int mat[][] = {
            { 1, 2, 3, 4 },
            { 5, 6, 7, 9 },
            { 9, 10, 11, 12 },
            { 13, 14, 15, 16 }
        };
        mainDiagonal(mat);
    }
}
```



Problem2.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Problem2- Print the Secondary Diagonal of n * n Matrix
public class Problem2 {

    //Time Complexity - O(n)
    //Aux Space Complexity - O(1)
    static void secondaryDiagonal(int mat[][]) {

        int n = mat.length;
        for (int i = 0, j = n - 1; i < n; i++, j--) {
            System.out.print(mat[i][j] + "\t");
        }
    }

    public static void main(String[] args) {
        int mat[][] = {
            { 1, 2, 3, 4 },
            { 5, 6, 7, 9 },
            { 9, 10, 11, 12 },
            { 13, 14, 15, 16 }
        };
        secondaryDiagonal(mat);
    }
}
```



Problem3A.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem3- Sum of Diagonal elements of n * n Matrix
public class Problem3A {

    // Time Complexity - O(2n)/O(n)
    // Aux Space Complexity - O(1)
    static void sumDiagonal(int mat[][]) {

        int n = mat.length;
        int sum = 0;

        for (int i = 0; i < n; i++) {
            sum = sum + mat[i][i];
        }

        for (int i = 0, j = n - 1; i < n; i++, j--) {
            if (i != j)
                sum = sum + mat[i][j];
        }
        System.out.println(sum);
    }

    public static void main(String[] args) {
        int mat[][] = {
            { 1, 2, 3, 4 },
            { 5, 6, 7, 9 },
            { 9, 10, 11, 12 },
            { 13, 14, 15, 16 }
        };
        sumDiagonal(mat);
    }
}
```



Problem3B.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Problem3- Sum of Diagonal elements of n * n Matrix

public class Problem3B {

    // Time Complexity - O(n)
    // Aux Space Complexity - O(1)
    static void sumDiagonal(int mat[][]){

        int n = mat.length;
        int sum = 0;

        for (int i = 0, j = n - 1; i < n; i++, j--) {
            if (i == j)
                sum = sum + mat[i][i];
            else
                sum = sum + mat[i][i] + mat[i][j];
        }
        System.out.println(sum);
    }

    public static void main(String[] args) {
        int mat[][] = {
            { 1, 2, 3, 4 },
            { 5, 6, 7, 9 },
            { 9, 10, 11, 12 },
            { 13, 14, 15, 16 }
        };

        sumDiagonal(mat);
    }
}
```



Problem4.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem4- Reverse the Rows of Matrix
public class Problem4 {
    // Time Complexity - O(n2)
    // Aux Space Complexity - O(1)
    static void reverseRows(int mat[][]){

        int rowCount = mat.length;

        for (int row = 0; row < rowCount; row++) {
            int start = 0;
            int end = mat[row].length - 1;
            while (start < end) {
                int temp = mat[row][start];
                mat[row][start] = mat[row][end];
                mat[row][end] = temp;

                start++;
                end--;
            }
        }
    }

    public static void main(String[] args) {

        int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 9 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };
        printMatrix(mat);
        reverseRows(mat);
        System.out.println("-----");
        printMatrix(mat);
    }
}
```



Problem5.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem5- Reverse the Columns of Matrix
public class Problem5 {
    // Time Complexity - O(n2)
    // Aux Space Complexity - O(1)
    static void reverseColumns(int mat[][]) {

        int colCount = mat.length;

        for (int col = 0; col < colCount; col++) {
            int start = 0;
            int end = colCount - 1;
            while (start < end) {
                int temp = mat[start][col];
                mat[start][col] = mat[end][col];
                mat[end][col] = temp;

                start++;
                end--;
            }
        }
    }

    public static void main(String[] args) {

        int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 9 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };
        printMatrix(mat);
        reverseColumns(mat);
        System.out.println("-----");
        printMatrix(mat);
    }
}
```



Problem6.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

//Problem6- Transpose of Matrix
public class Problem6 {

    // Time Complexity - O(n2)
    // Aux Space Complexity - O(1)
    static void transpose(int mat[][]) {

        int n = mat.length;
        for (int row = 0; row < n; row++) {

            for (int col = row + 1; col < n; col++) {
                int temp = mat[row][col];
                mat[row][col] = mat[col][row];
                mat[col][row] = temp;
            }
        }
    }

    public static void main(String[] args) {

        int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 9 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };
        printMatrix(mat);
        transpose(mat);
        System.out.println("-----");
        printMatrix(mat);
    }
}
```



Problem7.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem8- Rotate of Matrix to Clockwise by 90 degrees.
public class Problem7 {

    static void printMatrix(int mat[][]) {
        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                System.out.print(mat[i][j] + "\t");
            }
            System.out.println("");
        }
    }

    static void rotateRight90(int mat[][]) {
        transpose(mat);
        reverseRows(mat);
    }

    static void transpose(int mat[][]) {

        int n = mat.length;
        for (int row = 0; row < n; row++) {

            for (int col = row + 1; col < n; col++) {
                int temp = mat[row][col];
                mat[row][col] = mat[col][row];
                mat[col][row] = temp;
            }
        }
    }
}
```



```
static void reverseRows(int mat[][]) {

    int rowCount = mat.length;

    for (int row = 0; row < rowCount; row++) {
        int start = 0;
        int end = mat[row].length - 1;
        while (start < end) {
            int temp = mat[row][start];
            mat[row][start] = mat[row][end];
            mat[row][end] = temp;

            start++;
            end--;
        }
    }
}

public static void main(String[] args) {

    int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 9 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

    printMatrix(mat);
    rotateRight90(mat);
    System.out.println("-----");
    printMatrix(mat);
    rotateRight90(mat);
    System.out.println("-----");
    printMatrix(mat);
    rotateRight90(mat);
    System.out.println("-----");
    printMatrix(mat);
    rotateRight90(mat);
    System.out.println("-----");
    printMatrix(mat);

}
```



Problem8.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem8- Rotate of Matrix to Anti-Clockwise by 90 degrees.
public class Problem8 {

    static void printMatrix(int mat[][]) {
        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                System.out.print(mat[i][j] + "\t");
            }
            System.out.println("");
        }
    }

    static void rotateLeft90(int mat[][]) {
        transpose(mat);
        reverseColumns(mat);
    }

    static void transpose(int mat[][]) {

        int n = mat.length;
        for (int row = 0; row < n; row++) {

            for (int col = row + 1; col < n; col++) {
                int temp = mat[row][col];
                mat[row][col] = mat[col][row];
                mat[col][row] = temp;
            }
        }
    }
}
```



```
static void reverseColumns(int mat[][]) {

    int colCount = mat.length;

    for (int col = 0; col < colCount; col++) {
        int start = 0;
        int end = colCount - 1;
        while (start < end) {
            int temp = mat[start][col];
            mat[start][col] = mat[end][col];
            mat[end][col] = temp;

            start++;
            end--;
        }
    }
}

public static void main(String[] args) {

    int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 9 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

    printMatrix(mat);
    rotateLeft90(mat);
    System.out.println("-----");
    printMatrix(mat);
    rotateLeft90(mat);
    System.out.println("-----");
    printMatrix(mat);
    rotateLeft90(mat);
    System.out.println("-----");
    printMatrix(mat);
    rotateLeft90(mat);
    System.out.println("-----");
    printMatrix(mat);
}
```

Problem6A.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem6A- Transpose of m * n Matrix
//LeetCode - 867
public class Problem6A {

    // Time Complexity - O(m * n )
    // Aux Space Complexity - O(m * n )
    public static int[][] transpose(int[][] mat) {

        int rows = mat.length;
        int cols = mat[0].length;
        int result[][] = new int[cols][rows];

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                result[j][i] = mat[i][j];
            }
        }
        return result;
    }
    public static void main(String[] args) {
        // int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 9 },
        //                  { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };
        // int mat[][] = { { 1, 2, 3 }, { 4, 5, 6 } };
        int mat[][] = { { 1, 2 }, { 3, 4 }, { 5, 6 }, { 7, 8 } };
        printMatrix(mat);
        int result[][]=transpose(mat);
        System.out.println("-----");
        printMatrix(result);
    }
}
```



Problem9.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 9. Add two Matrices
public class Problem9 {

    static void printMatrix(int mat[][]) {

        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                System.out.print(mat[i][j] + "\t");
            }
            System.out.println("");
        }
    }

    // Time Complexity - O(rows * cols)
    //Time Complexity - O(rows * cols)
    static int[][] addMatrices(int a[][],int b[][]) {

        int rows = a.length;
        int cols=a[0].length;

        int c[][] = new int[rows][cols];

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                c[i][j] = a[i][j]+ b[i][j];
            }
        }

        return c;
    }
}
```



```
public static void main(String[] args) {

    int a[][] = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };
    int b[][] = { { 8, 7, 6 }, { 5, 4, 3 }, { 2, 1, 0 } };

    printMatrix(a);
    System.out.println("-----");
    printMatrix(b);
    System.out.println("-----");

    int c [][]= addMatrices(a,b);
    printMatrix(c);

}
```

Problem10.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 10. Multiply two Matrices
public class Problem10 {

    static void printMatrix(int mat[][]) {

        for (int i = 0; i < mat.length; i++) {
            for (int j = 0; j < mat[i].length; j++) {
                System.out.print(mat[i][j] + "\t");
            }
            System.out.println("");
        }

    }

}
```



```
// Time Complexity - O(rows * cols)
//Time Complexity - O(rows * cols)
static int[][] multiplyMatrices(int a[][], int b[][]) {

    int rows = a.length;
    int cols = a[0].length;

    int c[][] = new int[rows][cols];

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            for (int k = 0; k < rows; k++) {
                c[i][j] = c[i][j] + a[i][k] * b[k][j];
            }
        }
    }

    return c;
}

public static void main(String[] args) {

    int a[][] = { { 1, 2, 3 }, { 1, 2, 3 }, { 1, 2, 3 } };
    int b[][] = { { 1, 2, 3 }, { 1, 2, 3 }, { 1, 2, 3 } };

    printMatrix(a);
    System.out.println("-----");
    printMatrix(b);
    System.out.println("-----");

    int c[][] = multiplyMatrices(a, b);
    printMatrix(c);
}
}
```



Problem11.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem11- Snake Traversal of Matrix
public class Problem11 {

    static void snakeTraversal(int mat[][]) {

        int rows = mat.length;
        int cols = mat[0].length;

        for (int i = 0; i < rows; i++) {
            if (i % 2 == 0) {
                for (int j = 0; j < cols; j++) {
                    System.out.print(mat[i][j] + "\t");
                }
            } else {
                for (int j = cols - 1; j >= 0; j--) {
                    System.out.print(mat[i][j] + "\t");
                }
            }
            System.out.println("");
        }
    }

    public static void main(String[] args) {

        int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

        snakeTraversal(mat);

    }
}
```



Problem12.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Problem12- Boundary Traversal of Matrix
public class Problem12 {

    //Time Complexity - O( rows + cols + rows + cols)/O( rows + cols)
    //Aux Complexity - O(1)
    static void boundaryTraversal(int mat[][] ) {

        int rows = mat.length;
        int cols = mat[0].length;

        // First Row (Regular)
        for (int j = 0; j < cols; j++) {
            System.out.print(mat[0][j] + "\t");
        }

        // Last Column (Regular)
        for (int i = 1; i < rows; i++) {
            System.out.print(mat[i][cols-1] + "\t");
        }

        // Last Row (Reverse)
        for (int j = cols-2; j >=0; j--) {
            System.out.print(mat[rows - 1][j] + "\t");
        }

        // First Column (Reverse)
        for (int i = rows-2; i >=1; i--) {
            System.out.print(mat[i][0] + "\t");
        }
    }
}
```



```
public static void main(String[] args) {  
  
    int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };  
  
    boundaryTraversal(mat);  
  
}  
}
```

Problem13.java

```
package com.jlccindia.matrices;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
import java.util.ArrayList;  
import java.util.List;  
  
// Problem13- Spiral Traversal of Matrix  
//LeetCode - 54  
public class Problem13 {  
  
    // Time Complexity - O( rows * cols )  
    // Aux Space Complexity - O( rows * cols)  
    public static List<Integer> spiralOrder(int[][] mat) {  
  
        List<Integer> mylist = new ArrayList<Integer>();  
        int myrows = mat.length;  
        int mycols = mat[0].length;  
  
        int topRow = 0;  
        int rightCol = mycols - 1;  
        int bottomRow = myrows - 1;  
        int leftCol = 0;
```

```
while (mylist.size() < myrows * mycols) {
    // Top Row from Left to Right
    for (int col = leftCol; col <= rightCol; col++) {
        mylist.add(mat[topRow][col]);
    }

    // Right Col from Top to Bottom
    for (int row = topRow + 1; row <= bottomRow; row++) {
        mylist.add(mat[row][rightCol]);
    }

    if (topRow != bottomRow) {
        // Bottom Row from Right to Left
        for (int col = rightCol - 1; col >= leftCol; col--) {
            mylist.add(mat[bottomRow][col]);
        }
    }
    if (leftCol != rightCol) {
        // Left Col from Bottom to Top
        for (int row = bottomRow - 1; row > topRow; row--) {
            mylist.add(mat[row][leftCol]);
        }
    }
    topRow++;
    rightCol--;
    bottomRow--;
    leftCol++;
}
return mylist;
}

public static void main(String[] args) {
    int mat[][] = { { 1, 2, 3, 4, 5 }, { 6, 7, 8, 9, 10 },
                    { 11, 12, 13, 14, 15 }, { 16, 17, 18, 19, 20 }, { 21, 22, 23, 24, 25 } };
    List<Integer> mylist = spiralOrder(mat);
    System.out.println(mylist);
}
```



Problem14.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 14. Diagonal Traversal of Matrix
//LeetCode - 498
public class Problem14 {
    // Time Complexity - O( rows * cols )
    // Aux Space Complexity - O( rows * cols)
    public static int[] findDiagonalOrder(int[][] mat) {

        int myrows = mat.length;
        int mycols = mat[0].length;
        int result[] = new int[myrows * mycols];

        int curRow = 0;
        int curCol = 0;
        boolean up = true;
        int k=0;
        while (k != myrows * mycols) {
            if(up) {
                //Moving UP
                while(curRow>=0 && curCol<=mycols-1) {
                    result[k++] = mat[curRow][curCol];
                    curRow= curRow-1;
                    curCol = curCol + 1;
                }

                if(curCol==mycols) {
                    curCol = curCol - 1;
                    curRow= curRow+2;
                }else {
                    curRow= curRow+1;
                }
                up=false;
            }
        }
    }
}
```



```
        else {
            //Moving Down
            while(curCol>=0 && curRow<=myrows-1) {
                result[k++] = mat[curRow][curCol];
                curRow= curRow+1;
                curCol = curCol -1;
            }

            if(curRow==myrows) {
                curCol = curCol+2;
                curRow= curRow-1;
            }else {
                curCol = curCol+1;
            }
            up=true;
        }
    }
    return result;
}

public static void main(String[] args) {
    int mat[][] = { { 1, 2, 3, 4 }, { 5, 6, 7, 8 }, { 9, 10, 11, 12 }, { 13, 14, 15, 16 } };

    int [] result = findDiagonalOrder(mat);
    for(int x:result)
        System.out.println(x);
    }
}
```



Problem15.java

```
package com.jlcindia.matrices;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Problem15. Search 2-D Matrix
//LeetCode - 74
public class Problem15 {
    // Time Complexity - O(log(m * n))
    // Aux Space Complexity - O(1)
    public static boolean searchMatrix(int[][] mat, int element) {

        int myrows = mat.length;
        int mycols = mat[0].length;

        int topRow = 0;
        int rightCol = mycols-1;

        while(topRow<myrows && rightCol >=0) {
            if(element==mat[topRow][rightCol]) {
                return true;
            } else if(element<mat[topRow][rightCol]) {
                rightCol--;
            } else if(element>mat[topRow][rightCol]) {
                topRow++;
            }
        }
        return false;
    }
    public static void main(String[] args) {
        int mat[][] = { { 1,3,5,7 }, { 10,11,16,20 }, {23,30,34,60}, { 62 ,65,70,75} };
        int element = 75;
        boolean result = searchMatrix(mat,element);
        System.out.println(result);
    }
}
```